



★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Spring Boot Actuator



Niraj Kumar · [Follow](#)

2 min read · Mar 2, 2024



Spring Boot Actuator is a set of production-ready features that help you monitor and manage your application. It provides a wide range of built-in endpoints and additional features for gathering insights into the application's runtime behavior, health, metrics, and more. Below are some key features and concepts related to Spring Boot Actuator:

Key Features:

1. Health Endpoint:

- The `/actuator/health` endpoint provides information about the application's health. It checks the status of various components, such as the database, disk space, and custom health indicators.

2. Info Endpoint:

- The `/actuator/info` endpoint allows you to expose arbitrary application information. You can customize this information in the `application.properties` or `application.yml` file.

3. Metrics Endpoint:

- The `/actuator/metrics` endpoint exposes a variety of metrics about the application, including JVM metrics, garbage collection, thread usage, and custom metrics.

4. Environment Endpoint:

- The `/actuator/env` endpoint displays information about the application's environment properties and configuration.

5. Auditing Endpoint:

- The `/actuator/auditevents` endpoint provides information about authentication success and failure events.

6. Application Endpoints:

- Spring Boot Actuator includes various other endpoints for application-specific details, such as `/actuator/beans`, `/actuator/mappings`, and `/actuator/conditions`.

7. Shutdown Endpoint:

- The `/actuator/shutdown` endpoint allows you to gracefully shut down the application.

8. Custom Endpoints:

- You can create custom endpoints by implementing the `Endpoint` interface and extending the `AbstractEndpoint` class.

Configuration:

To enable Spring Boot Actuator, you need to include the following dependency in your `pom.xml` file (for Maven projects):

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Security:

Spring Boot Actuator endpoints may expose sensitive information, so it's crucial to secure them. By default, they are protected with basic authentication. You can customize security settings in the `application.properties` or `application.yml` file. For example:

```
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
management.endpoint.health.roles=ACTUATOR_ADMIN
management.endpoint.metrics.enabled=true
management.endpoint.metrics.roles=ACTUATOR_ADMIN
```

Customizing Endpoints:

You can customize and extend Spring Boot Actuator by providing additional configuration, creating custom health indicators, and implementing custom endpoints.

Example Endpoint Configuration:

```
management:
  endpoints:
    web:
      exposure:
        include: "*"
    endpoint:
      health:
        show-details: always
      metrics:
        enabled: true
    endpoint:
      metrics:
        roles: ACTUATOR_ADMIN
```

Accessing Endpoints:

Once your application is running, you can access the actuator endpoints using HTTP requests. For example:

- Health Endpoint: <http://localhost:8080/actuator/health>
- Metrics Endpoint: <http://localhost:8080/actuator/metrics>
- Custom Endpoint: <http://localhost:8080/actuator/custom>

Spring Boot Actuator provides valuable insights into your application's runtime behavior, making it easier to monitor, manage, and troubleshoot issues in production. For detailed information and customization options, refer to the official Spring Boot Actuator documentation.